GIT HUB LINK :
https://github.com/astarohan/AIML-LAB-PEC

NAME:ROHAN RESHI

PRN:202401040034

BATCH:A1-6

LAB ASSIGNMENT 3

PROBLEM SOLVING USING SEARCH TECHNIQUES

Problem: Snake and Ladder using Search Techniques

AI SEARCH ALGORITHMS USING SNAKE AND LADDER

Uninformed Search and Informed Search

Algorithms Implemented:

1. Depth First Search (DFS)
2. Breadth First Search (BFS)
3. Depth Limited Search (DLS)
4. Iterative Deepening Search (IDS)
5. A* Search
6. Best First Search
7. Hill Climbing
8. Genetic Algorithm

AIM

To implement and study different Artificial Intelligence search algorithms using the Snake and Ladder game as a state-space search

problem.

The algorithms implemented are:

- Depth First Search (DFS)
- Breadth First Search (BFS)
- Depth Limited Search (DLS)
- Iterative Deepening Search (IDS)
- A* Search
- Best First Search
- Hill Climbing
- Genetic Algorithm

The main objective is to find a path from the starting position 1 to the winning position 100 using different search techniques.

PROBLEM STATEMENT

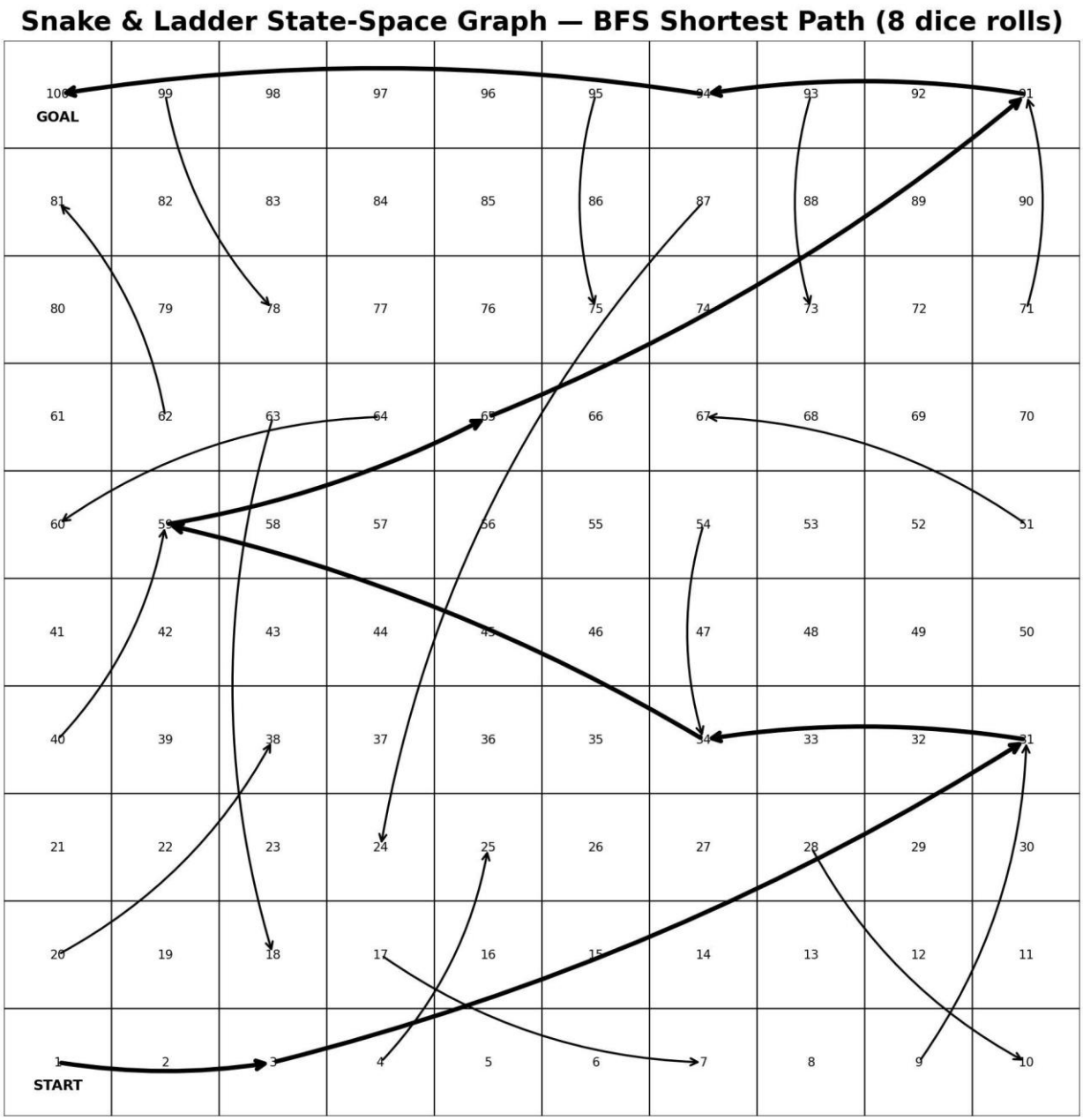
Snake and Ladder can be represented as a state-space search problem.

The game board contains positions from 1 to 100.

- Initial State: Position 1
- Goal State: Position 100
- Actions: Rolling a dice from 1 to 6
- State: Current position of the player
- Transition: The player moves according to the dice value
- Snakes and ladders modify the final position
- Cost of each dice roll: 1

The objective is to find a path from position 1 to position 100 using different AI search algorithms.

The path cost represents the number of dice rolls required to reach the winning position.




```
snake_ladder = {
    4: 25,
    9: 31,
    17: 7,
    20: 38,
    28: 10,
    40: 59,
    51: 67,
    54: 34,
    62: 81,
    63: 18,
    64: 60,
    71: 91,
    87: 24,
```

```
        93: 73,
        95: 75,
        99: 78
    }

    def create_board():
        board = {}

        for position in range(1, 101):
            board[position] = []

            for dice in range(1, 7):
                next_position = position + dice

                if next_position <= 100:

                    if next_position in snake_ladder:
                        next_position = snake_ladder[next_position]

                    board[position].append(next_position)

        return board

    board = create_board()

    print("Board created successfully!")
    print("Moves from position 1:", board[1])
```

```
Board created successfully!
Moves from position 1: [2, 3, 25, 5, 6, 7]
```

UNINFORMED SEARCH

Uninformed search algorithms do not use additional information about how close a state is to the goal.

They search the state space using only the information available from the problem definition.

The following uninformed search algorithms are implemented:

1. Depth First Search (DFS)

2. Breadth First Search (BFS)
3. Depth Limited Search (DLS)
4. Iterative Deepening Search (IDS)

These algorithms do not use a heuristic function.

Depth First Search (DFS)

Depth First Search is an uninformed search algorithm that explores a path as deeply as possible before backtracking.

It uses a Stack data structure and follows LIFO (Last In, First Out).

```
def depth_first(board, source, goal):

    if source not in board or goal not in board:
        return None

    stack = [source]
    visited = set([source])
    parent = {source: None}

    while stack:

        current = stack.pop()

        if current == goal:
            print("Goal Found")

            path = []

            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()
            return path

        for neighbor in board[current]:

            if neighbor not in visited:
```

```
                    visited.add(neighbor)
                    parent[neighbor] = current
                    stack.append(neighbor)

        return None


route = depth_first(board, 1, 100)

if route is None:
    print("Goal not found")
else:
    print("Route to the winning position is found")
    print(" -> ".join(map(str, route)))
```

```
Goal Found
Route to the winning position is found
1 -> 7 -> 13 -> 19 -> 24 -> 30 -> 36 -> 42 -> 48 -> 53 -> 58 ->
```

Output Explanation

DFS successfully reaches the goal position 100.

The algorithm explores one branch deeply before trying another branch. Therefore, the path found can be longer than the minimum path.

The obtained path requires 15 dice rolls. Unlike BFS, DFS does not guarantee the minimum-roll path.

Breadth First Search (BFS)

Breadth First Search is an uninformed search algorithm that explores the nodes level by level.

It uses a Queue data structure and follows FIFO (First In, First Out).

```
def breadth_first(board, source, goal):
```

```
    if source not in board or goal not in board:
        return None

    queue = [source]
    visited = set([source])
    parent = {source: None}

    while queue:

        current = queue.pop(0)

        if current == goal:
            print("Goal is found")

            path = []

            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()
            return path

        for child in board[current]:

            if child not in visited:
                visited.add(child)
                parent[child] = current
                queue.append(child)

    return None


route = breadth_first(board, 1, 100)

if route is None:
    print("Goal not found")
else:
    print("Route to the winning position is found")
    print(" -> ".join(map(str, route)))
```

```
Goal is found
Route to the winning position is found
1 -> 3 -> 31 -> 34 -> 59 -> 65 -> 91 -> 94 -> 100
```

Output Explanation

BFS successfully reaches the goal position 100.

The algorithm explores the board level by level using a queue. Since every dice roll has the same cost, BFS finds a path with the minimum number of dice rolls.

The obtained path requires 8 dice rolls.

Depth Limited Search (DLS)

Depth Limited Search is a variation of Depth First Search that imposes a depth limit on the search.

It avoids searching beyond the specified depth.

```
def depth_limited_search(board, start, goal, limit):

    def dfs(node, depth, path):

        if node == goal:
            return path

        if depth == limit:
            return None

        for neighbour in board[node]:

            if neighbour not in path:

                result = dfs(
                    neighbour,
                    depth + 1,
                    path + [neighbour]
                )

                if result is not None:
                    return result
```

```
            return None

        return dfs(start, 0, [start])


path = depth_limited_search(board, 1, 100, 10)

if path is None:
    print("Goal not found")
    print("Maybe you should increase the limit")
else:
    print("Route to the winning position is found")
    print(" -> ".join(map(str, path)))
```

```
Route to the winning position is found
1 -> 2 -> 3 -> 25 -> 29 -> 34 -> 59 -> 65 -> 91 -> 94 -> 100
```

Output Explanation

DLS successfully reaches the goal position 100 within the given depth limit.

The algorithm works like DFS but does not explore beyond the specified depth. In this case, the depth limit is 10, so the search is restricted to paths within this limit.

The obtained path requires 10 dice rolls.

Iterative Deepening Search (IDS)

Iterative Deepening Search combines Depth First Search and Breadth First Search.

It performs Depth Limited Search repeatedly with increasing depth limits until the goal is found.

```
def iterative_deepening_search(board, start, goal, max_depth):
```

```
        for depth in range(max_depth + 1):

            path = depth_limited_search(
                board,
                start,
                goal,
                depth
            )

            if path is not None:
                return path

        return None


    iterative_path = iterative_deepening_search(
        board,
        1,
        100,
        20
    )

    if iterative_path is None:
        print("Goal not found")
    else:
        print("Route to the winning position is found")
        print(" -> ".join(map(str, iterative_path)))
```

```
Route to the winning position is found
1 -> 3 -> 31 -> 34 -> 59 -> 65 -> 91 -> 94 -> 100
```

Output Explanation

IDS successfully reaches the goal position 100 by repeatedly increasing the depth limit.

It first searches with a small depth limit and then increases the limit until the goal is found.

The obtained path requires 8 dice rolls. IDS combines the depth-limited approach of DFS with increasing depth limits.

Heuristic Function

A heuristic function estimates the cost from the current position to the goal.

For Snake and Ladder:

h(n) = 100 - current position

```
def heuristic(position):
    return 100 - position
```

INFORMED SEARCH

Informed search algorithms use additional information about the problem to guide the search toward the goal.

This additional information is provided using a heuristic function.

A heuristic estimates how close a current state is to the goal.

The informed search algorithms used in this project are:

1. A* Search
2. Best First Search

The project also uses heuristic-based local search methods:

3. Hill Climbing
4. Genetic Algorithm

A* Search

A* Search is an informed search algorithm that uses the actual cost and heuristic cost to find a path.

The evaluation function is:

f(n) = g(n) + h(n)

where:

g(n) = actual cost from the start h(n) = estimated cost to the goal

```
def a_star_search(board, start, goal):

    open_array = set([start])
    close_array = set()

    node_distance = {start: 0}
    parent = {start: start}

    while len(open_array) > 0:

        current_node = None
        lowest_f_cost = float('inf')

        for node in open_array:

            f_cost = (
                node_distance[node]
                + heuristic(node)
            )

            if f_cost < lowest_f_cost:
                lowest_f_cost = f_cost
                current_node = node

        if current_node is None:
            print("PATH DOES NOT EXIST!")
            return None

        if current_node == goal:

            path = []

            while parent[current_node] != current_node:
                path.append(current_node)
                current_node = parent[current_node]

            path.append(start)
            path.reverse()
```

```
            print("A* Path:")
            print(*path, sep=" -> ")
            print("Number of dice rolls:", len(path) - 1)

            return path

        for neighbour in board[current_node]:

            new_distance = node_distance[current_node] + 1

            if neighbour not in open_array and neighbour not in

                open_array.add(neighbour)
                parent[neighbour] = current_node
                node_distance[neighbour] = new_distance

            elif (
                neighbour in node_distance
                and new_distance < node_distance[neighbour]
            ):

                node_distance[neighbour] = new_distance
                parent[neighbour] = current_node

                if neighbour in close_array:
                    close_array.remove(neighbour)
                    open_array.add(neighbour)

        open_array.remove(current_node)
        close_array.add(current_node)

    print("PATH DOES NOT EXIST!")
    return None


shortest_path = a_star_search(board, 1, 100)
```

```
A* Path:
1 -> 25 -> 31 -> 37 -> 59 -> 81 -> 86 -> 92 -> 98 -> 100
Number of dice rolls: 9
```

Output Explanation

A* successfully reaches the goal position 100 using both the actual path cost and the heuristic value.

The algorithm calculates:

f(n) = g(n) + h(n)

where g(n) is the actual cost and h(n) estimates the remaining distance to the goal.

The obtained path requires 9 dice rolls. The heuristic guides the search towards positions closer to 100.

Best First Search

Best First Search is an informed search algorithm that selects the node that appears closest to the goal using a heuristic function.

It mainly uses h(n) to decide which node to explore next.

```
def best_first_search(board, start, goal):

    open_list = [start]
    closed_list = set()

    parent = {start: None}

    while open_list:

        current = min(
            open_list,
            key=lambda node: heuristic(node)
        )

        if current == goal:

            path = []
```

```
            while current is not None:
                path.append(current)
                current = parent[current]

            return path[::-1]

        open_list.remove(current)
        closed_list.add(current)

        for neighbour in board[current]:

            if (
                neighbour not in closed_list
                and neighbour not in open_list
            ):

                parent[neighbour] = current
                open_list.append(neighbour)

    return None


path = best_first_search(board, 1, 100)

if path:
    print("Best First Search Path:")
    print(" -> ".join(map(str, path)))
else:
    print("Path not found")
```

```
Best First Search Path:
1 -> 25 -> 31 -> 37 -> 59 -> 81 -> 86 -> 92 -> 98 -> 100
```

Output Explanation

Best First Search successfully reaches the goal position 100.

The algorithm selects the position with the lowest heuristic value and therefore tries to move towards positions closer to 100.

The obtained path requires 9 dice rolls. Unlike A*, Best First Search mainly uses the heuristic value to select the next node.

Hill Climbing

Hill Climbing is a local search algorithm that repeatedly moves towards a better neighbouring solution.

It is a greedy approach and can get stuck at a local optimum.

```
import random

def play_game(rolls):

    position = 1
    path = [1]

    for roll in rolls:

        next_position = position + roll

        if next_position > 100:
            break

        position = next_position

        if position in snake_ladder:
            position = snake_ladder[position]

        path.append(position)

        if position == 100:
            return position, path

    return position, path


def hill_climbing(
    max_rolls=30,
    max_iterations=5000
):
```

```
current = [
    random.randint(1, 6)
    for _ in range(max_rolls)
]

best_solution = None
best_path = None
best_cost = float('inf')

for iteration in range(max_iterations):

    neighbour = current.copy()

    index = random.randint(
        0,
        max_rolls - 1
    )

    neighbour[index] = random.randint(1, 6)

    neighbour_position, neighbour_path = play_game(
        neighbour
    )

    if neighbour_position == 100:

        neighbour_cost = len(neighbour_path) - 1

        if neighbour_cost < best_cost:

            best_solution = neighbour
            best_path = neighbour_path
            best_cost = neighbour_cost

    current_position, current_path = play_game(current)

    if neighbour_position > current_position:
        current = neighbour

print("=" * 50)
print("HILL CLIMBING - BEST SOLUTION")
print("=" * 50)
```

```
        if best_solution is not None:

            print("Best Path:", best_path)
            print("Number of Dice Rolls:", best_cost)
            print("Path Cost:", best_cost)

        else:
            print("No solution reached 100.")

        return best_path, best_cost


hill_path, hill_cost = hill_climbing()
```

```
==================================================
HILL CLIMBING - BEST SOLUTION
==================================================
Best Path: [1, 2, 7, 12, 7, 31, 37, 38, 59, 81, 86, 91, 96, 100]
Number of Dice Rolls: 13
Path Cost: 13
```

Output Explanation

Hill Climbing searches for a better solution by modifying the current sequence of dice rolls.

It accepts a neighbouring solution when it moves closer towards the goal and keeps searching for an improved valid path.

The output shows the best valid path found during the search. The path cost can vary between runs because the algorithm uses random neighbours.

Hill Climbing does not guarantee the globally shortest path.

Genetic Algorithm

A Genetic Algorithm is a metaheuristic inspired by natural selection.

It searches for solutions using a population and applies selection, crossover and mutation to improve the solutions.

```
import random

def play_game(chromosome):

    position = 1
    path = [1]
    dice_rolls = []

    for roll in chromosome:

        next_position = position + roll

        if next_position > 100:
            break

        position = next_position
        dice_rolls.append(roll)

        if position in snake_ladder:
            position = snake_ladder[position]

        path.append(position)

        if position == 100:
            break

    return position, path, dice_rolls


def fitness(chromosome):

    position, path, dice_rolls = play_game(chromosome)

    if position == 100:

        path_cost = len(path) - 1

        return 10000 - path_cost

    return position
```

```
def genetic_algorithm(
    population_size=50,
    chromosome_length=30,
    max_iterations=5000
):

    population = []

    for _ in range(population_size):

        chromosome = [
            random.randint(1, 6)
            for _ in range(chromosome_length)
        ]

        population.append(chromosome)

    solution_number = 0

    best_cost = float('inf')
    best_solution = None

    improving_solutions = []

    for iteration in range(1, max_iterations + 1):

        results = []

        for chromosome in population:

            solution_number += 1

            position, path, dice_rolls = play_game(
                chromosome
            )

            results.append(
                (
                    position,
                    path,
                    dice_rolls,
                    chromosome
```

```
                )
            )

            if position == 100:

                path_cost = len(path) - 1

                if path_cost < best_cost:

                    best_cost = path_cost

                    best_solution = {
                        "solution_number": solution_number,
                        "iteration": iteration,
                        "path": path.copy(),
                        "dice_rolls": dice_rolls.copy(),
                        "path_cost": path_cost
                    }

                    improving_solutions.append(
                        best_solution.copy()
                    )

        results.sort(
            key=lambda x: fitness(x[3]),
            reverse=True
        )

        parent1 = results[0][3]
        parent2 = results[1][3]

        new_population = [
            parent1,
            parent2
        ]

        while len(new_population) < population_size:

            crossover_point = random.randint(
                1,
                chromosome_length - 1
            )

            child = (
```

```
                parent1[:crossover_point]
                +
                parent2[crossover_point:]
            )

            if random.random() < 0.10:

                mutation_index = random.randint(
                    0,
                    chromosome_length - 1
                )

                child[mutation_index] = random.randint(
                    1,
                    6
                )

            new_population.append(child)

        population = new_population

    if best_solution is None:

        print("No solution reached 100.")
        return None

    print("=" * 60)
    print("IMPROVING SOLUTIONS")
    print("=" * 60)

    for solution in improving_solutions:

        print()
        print(
            "Solution Number:",
            solution["solution_number"]
        )

        print(
            "Iteration:",
            solution["iteration"]
        )

        print(
```

```
            "Path:",
            solution["path"]
        )

        print(
            "Path Cost:",
            solution["path_cost"]
        )

print()
print("=" * 60)
print("BEST SOLUTION")
print("=" * 60)

print()
print(
    "Best Solution Number:",
    best_solution["solution_number"]
)

print(
    "Best Iteration:",
    best_solution["iteration"]
)

print(
    "Best Path:",
    best_solution["path"]
)

print(
    "Best Path Cost:",
    best_solution["path_cost"]
)

print(
    "Minimum Path Cost:",
    best_cost
)

print(
    "Number of Dice Rolls:",
    best_solution["path_cost"]
)
```

```
    return best_solution["path"], best_cost


best_path, best_cost = genetic_algorithm(
    population_size=50,
    chromosome_length=30,
    max_iterations=5000
)
```

```
============================================================
IMPROVING SOLUTIONS
============================================================

Solution Number: 1
Iteration: 1
Path: [1, 3, 6, 12, 16, 21, 27, 30, 32, 34, 38, 42, 46, 67, 73,
Path Cost: 30

Solution Number: 36
Iteration: 1
Path: [1, 3, 5, 31, 34, 35, 59, 81, 83, 88, 92, 98, 78, 83, 84,
Path Cost: 20

Solution Number: 41
Iteration: 1
Path: [1, 7, 31, 37, 41, 42, 48, 67, 73, 74, 80, 85, 88, 91, 94,
Path Cost: 15

Solution Number: 311
Iteration: 7
Path: [1, 7, 31, 37, 41, 42, 48, 34, 59, 60, 66, 91, 94, 97, 100
Path Cost: 14

Solution Number: 915
Iteration: 19
Path: [1, 7, 31, 37, 41, 42, 48, 34, 59, 60, 66, 91, 94, 100]
Path Cost: 13

============================================================
BEST SOLUTION
============================================================

Best Solution Number: 915
Best Iteration: 19
Best Path: [1, 7, 31, 37, 41, 42, 48, 34, 59, 60, 66, 91, 94, 10
Best Path Cost: 13
```

```
Minimum Path Cost: 13
Number of Dice Rolls: 13
```

Output Explanation

The Genetic Algorithm generates a population of possible dice roll sequences and improves them through selection, crossover and mutation.

Only improving solutions are displayed, showing how the path cost becomes better during the search.

The final output shows the best solution found, including its solution number, iteration, path and minimum path cost.

Since random operations are used, the result can change between different runs.

Conclusion

Different AI search algorithms were implemented on the Snake and Ladder problem.

DFS, BFS, DLS and IDS are uninformed search algorithms, while A* and Best First Search are informed search algorithms.

Hill Climbing and Genetic Algorithm are optimization-based search techniques.